



DATA 604:
DATA REPRESENTATION AND MODELING

FINAL PROJECT - SPRING 2023
May 8, 2023

Analysis of Grayscale CIFAR10 Dataset Using
Different Data Representations and kNN

Bishal Regmi

SCIENCE ACADEMY
UNIVERSITY OF MARYLAND, COLLEGE PARK

Contents

1. Introduction	3
2. Dataset	3
3. Raw Data	5
3.1. Raw Data Analysis	5
3.2. Finding the Best K Value	6
3.3. Finding the Best N Value	7
3.4. kNN with Raw Data and Results	8
4. Principal Component Analysis	9
4.1. PCA Data Analysis	9
4.2. kNN with PCA Data and Results	11
5. Kernel Principal Component Analysis	12
5.1. kPCA Data Analysis	13
5.2. kNN with kPCA Data and Results	14
6. t-SNE	16
6.1. t-SNE Data Analysis	16
6.2. kNN with t-SNE Data and Results	17
7. Comparative Analysis	18
8. Conclusion	19
A. Appendix	21
A.1. Regular Data Extra Figures and Tables	21
A.2. PCA Extra Figures and Tables	21
A.3. kPCA Extra Figures and Tables	23
A.4. t-SNE Extra Figures and Tables	24

1. Introduction

This report presents a thorough comparative analysis of various data representation methods applied to the grayscale CIFAR10 Dataset. The objective is to evaluate the effectiveness of these representation methods, including raw data, Principal Component Analysis (PCA), Kernel Principal Component Analysis (kPCA), and t-Distributed Stochastic Neighbor Embedding (t-SNE), in enhancing the performance of a nonlinear classifier, specifically the k-Nearest Neighbors (kNN) algorithm. The report is structured as follows: first, the report provides a detailed description of the CIFAR10 Dataset in grayscale, followed by an in-depth examination of each data representation method. In conjunction with the analysis of each method, it utilizes the kNN classifier for training, validation, and testing where valid, discussing the outcomes and highlighting the advantages and drawbacks of each representation technique. Upon completing the individual analyses, it proceeds with a comprehensive comparison of the methods, assessing their relative merits with two distinct measures of success: class accuracy and global/average accuracy. By conducting this comparative analysis, this report aims to draw meaningful conclusions about the optimal data representation method for the CIFAR10 Dataset in grayscale, considering the performance of the kNN classification algorithm.

2. Dataset

The CIFAR10 dataset is a widely recognized and utilized benchmark dataset for image classification tasks [2]. Comprising 60,000 32x32 color images in total, the dataset is divided into 10 distinct classes, each containing 6,000 images [2]. These classes represent various everyday objects and living beings, including airplanes, automobiles, birds, cats, deer, dogs, frogs, horses, ships, and trucks. In Figure 1, all the classes and their associated labels are listed. In Figure 2, the first 10 images from the dataset are shown. All the images in the dataset come sideways, and this is not altered, but when displayed might be visually altered to make it easier to see.

Label	Description
0	Airplane
1	Automobile
2	Bird
3	Cat
4	Deer
5	Dog
6	Frog
7	Horse
8	Ship
9	Truck

Figure 1: Classes and their labels

For the purpose of this analysis, the CIFAR10 dataset is converted to grayscale. Figure 3 shows an image from the frog class converted from color to gray. This transformation reduces the complexity of the dataset by removing the color information and representing each image using a single channel instead of three. This helps reduce the number of features from 3072 down to 1028, reducing the computational and space cost of the analysis. The size adjustment is required due to the low computational power of the computer the analysis is conducted on. In addition to reducing the number of features, kPCA requires reducing the number of observations due to computational cost issues. This is further explained in a later section.

Moreover, the CIFAR10 dataset is renowned for its classification challenges. As noted in a Stanford course publication, the kNN algorithm achieves a maximum overall accuracy of ap-

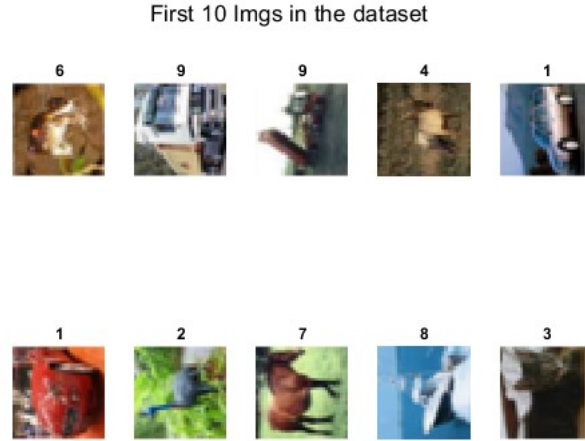


Figure 2: First 10 Images from the Dataset

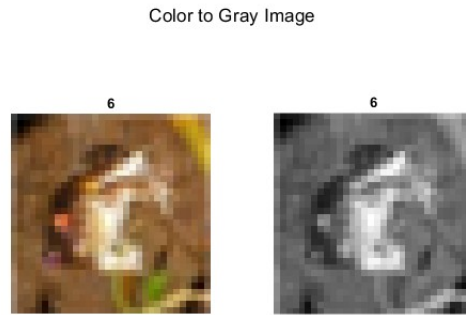


Figure 3: Image Converted from Color to Gray

proximately 40 percent when applied to the color dataset [1]. It is worth mentioning that this 40 percent accuracy was obtained using the full-color dataset, which contains more features than the grayscale version utilized in this analysis [1]. Given that two-thirds of the features are eliminated in the grayscale data, the anticipated overall accuracy throughout this study is expected to be relatively low and most likely lower than 40 percent. Despite this, this project proceeds with an in-depth examination of the raw grayscale data, providing valuable insights into the effectiveness of various data representation methods in the context of this complex dataset. To properly apply methods like PCA, kPCA, and t-SNE, the data must be carefully pre-processed. In this case, with image data, two of them are checking for image orientation and data normalization.

The orientation of the images is important as the orientation can affect the results in processes like PCA, which relies on finding the directions of maximum variation in the data. With that, the orientation of the data can affect the relative importance of different features. In Figure 4, the first images from each of the ten classes are shown. Additionally, the rest of the images are similarly oriented, where all the images are sideways. Given that all the images are oriented in the same direction, no additional step is required.

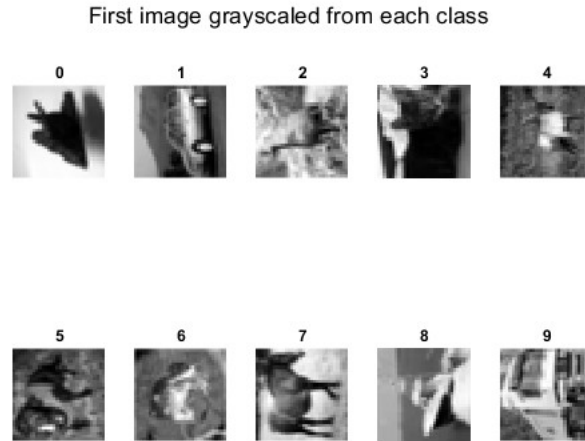


Figure 4: Images from Each Class

Another thing to take note of in the data processing step is whether to normalize or standardize the data or not. It is generally recommended to normalize or standardize when doing PCA or any other data representation methods that can be affected by outliers. PCA benefits from normalization, sometimes called standardization or scaling, because it assures that all of the dataset's variables have the same scale, making their contributions to the principal components comparable. In this case, the data points are adjusted to be between 0 and 1 by dividing the pixel value, which ranges from 0 to 255, by 255, and all the pixel values are from the same scale. In this case, it is sufficient not to normalize but just center, and the functions that require it already do that. There is the possibility of minimal gain in performance and better analysis when normalizing, but some meaning might be lost in normalizing each pixel value separately. With that, the data is not normalized.

3. Raw Data

This section looks at the raw data and does some minor analysis to come up with some assumptions, and then uses it for kNN to get a baseline on how well the raw data performs. The results are also explained and later used for comparative analysis.

3.1. Raw Data Analysis

To begin with, the images in the dataset are sideways when displayed, as discussed earlier. This can be seen in Figure 4. The visually rotated and flipped version of the image is in Figure 5 to make it easier to see. The actual data is not altered, but whenever a rotated image is shown the images are just a rotated version of the data.

The main information to extract from Figures 2, 4, and 5 is that these images are hard for humans to differentiate confidently. Once, the color is removed, it is further difficult. For example, class 5 in Figure 5 is difficult to decipher even knowing what the class labels mean, similar to classes 2 and 3 in the same figure. Unlike datasets like Fashion MNIST and Digit MNIST, where all the objects or digits in the images are in the same orientation without any background, these images contain a lot of extra information besides the objects or beings that are being classified. This is further evident when doing a class sample mean. Figure 6 shows the sample mean of all the classes, and unlike the Fashion MNIST and Digit MNIST datasets, these means can not be

First image grayscale from each class in 2 by 5 grid (Rotated)

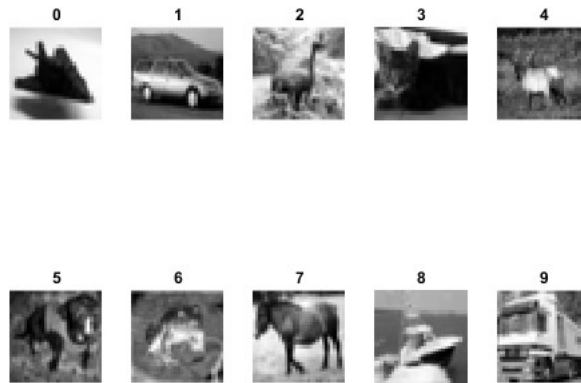


Figure 5: Images from Each Class Rotated

connected back to their classes. So algorithms that use distance to determine similarity and use it for classification, like kNN, are bound to have a difficult time differentiating each of the classes. With that, the next step is to solidify further the assumption using kNN.

Sample Mean of each class

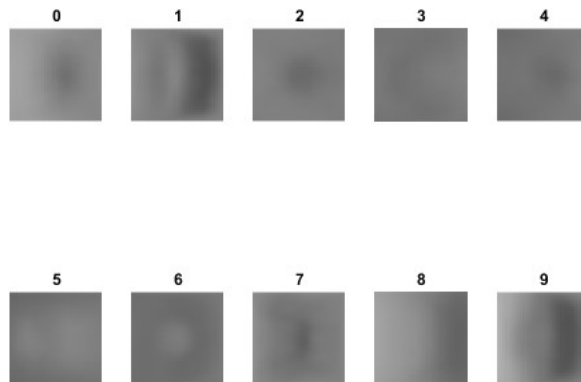


Figure 6: Class Sample Mean

3.2. Finding the Best K Value

Before continuing to kNN, this analysis starts by finding the best k value to use for kNN. The best split is not tested yet, so a generally valid split size is used. Picking a train-validate-test split of 80-10-10, the dataset with 60000 data points is split into 48000 for the training set, 6000 for the validation set, and 6000 for the test set with an equal number of all the+ classes in each set. It is also worth noting that in order to split the data, an N value is picked, which is used to split train and test. Then, the train set containing N is split into the training and validation set by

taking as many observations as there are in the test set from the end of the training set. The test and validation set sizes are always constant as per the general guideline. This is held constant for all the values of k tested. The train and validation sets are used to find the best k value, and finally, the accuracy is confirmed with the test set. Going into how the k NN models are created, the k values differ on each run, but the distance metric is kept constant on Euclidean. With that, k values ranging from 5 to 30 are tested. Figure 7 shows the validation global accuracy for each run. The range of the y values in the plot is between 0.25 to 0.30.

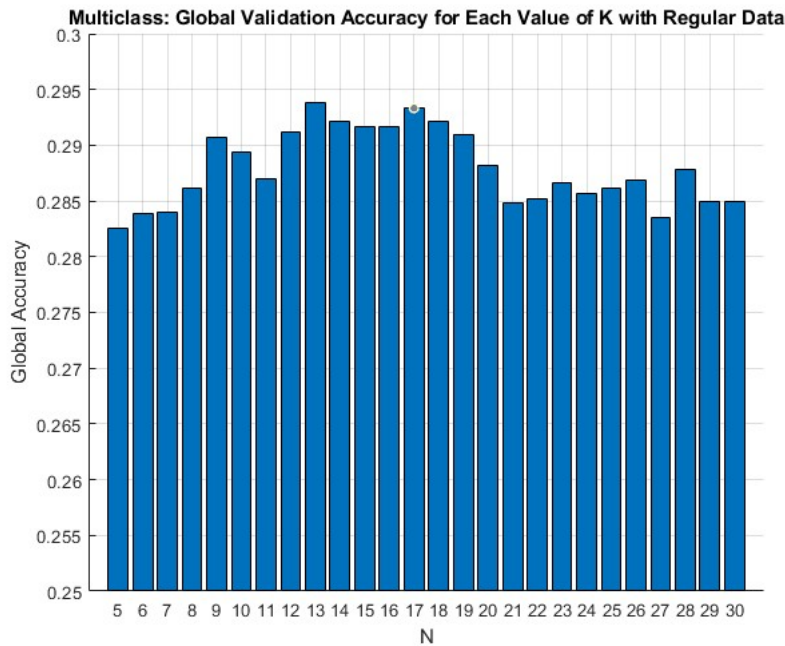


Figure 7: Validation Score for Each K Value

The results are a bit interesting. The general idea for picking the best k value would be to pick the smallest values where the performance is the highest, and the performance decreases after that. The two options there are 9 and 13, as 9 is the initial value that had the highest, and then the performance drops but again goes up to the max performance at 13, and then after that, it continues to drop. With that, a k value of 13 is picked. The test global accuracy is 0.2918 with a k of 13.

3.3. Finding the Best N Value

Now that the best k is known to be 13, the best split must be determined. As discussed earlier, when setting up the analysis for the best k value, the method by which the splits are generated is kept the same. The only thing adjusted in this analysis is that the N values differ for each run, while the k value is kept the same at 13. Again to explain what the N values mean, the N value is the size of the train and validation set combined, containing N observations from each class. In the test class, there are $6000 - N$ observations from each class. In the validation set, the last $6000 - N$ observations from each class are taken. With that, the test and validation sets have the same size. This use of the N value is the same for the rest of the other analysis as well. With that, N values from 4000 all the way to 5500, are tested with an interval of 50 for a total of 31 different values. Figure 8 shows the training and validation global accuracy.

It is clear that with great N value, the plots of training and validation accuracies both continue to grow. The validation accuracy does go down at 5450 but goes up again. With that, the great

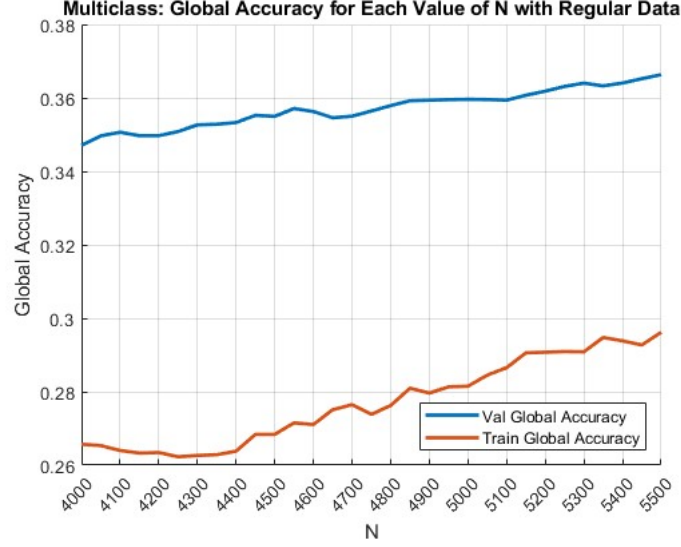


Figure 8: Global Accuracy for Validation and Training Set With Different N Values

the N values, the better the results are bound to be. This leads to a split of approximately 84-8-8. The test accuracy at N equal to 5500 is 0.289. This is slightly lower than the test accuracy achieved from the slightly lower N value when testing the best k values. There might be more merit in analyzing this in further research, but for now, the next step is to actually conduct the kNN with all the values above and discuss the results in terms of global and class accuracy.

3.4. kNN with Raw Data and Results

To recap how the model is set up, the training split, as used in the past testing, is used along with only the test set to complete this portion. So, N is set to 5500, and where the train set has 5000 observations from each class and 500 from each class in the test. The k value is 13, and the distance metric is Euclidean. The resulting global test accuracy is 0.2892, and, respectively, for each class, the accuracy is stored in Table 2 in the Appendix. Figure 9 shows the confusion matrix created from the results.

Overall, the global accuracy is low, as expected from what the Stanford notes explained, and as assumed for the grayscale data with fewer features [1]. The main analysis will come from comparing this result to that of the other data representation method. Looking further into the class accuracy as shown in Table 2, the best class is the ship class (8) with the highest accuracy of 0.6340, followed by deer (4) with an accuracy of 0.4660. The next two classes are airplane (0) and bird (2), which is very interesting because the two things that can fly have a similar score. After that, other than the frog class (6) with an accuracy of 0.2480, the other classes, automobile (1), cat (3), dog (5), and horse (7), have similar accuracies under 0.2 and above 0.1, with the truck class (9) having the least accuracy of 0.1240. Looking at the confusion matrix in Figure 9, class 8 seems to have both a lot of false positives and a lot of true positives but not as many false negatives. It seems the class recall is high, but precision is very low. But the overall pattern is that all the classes are confused with each other, and the overall error is high lots of false positives and false negatives. The classes that did the best, like 0, 2, 4, and 8, were only scoring high due to the model thinking most of the observations were those classes, and with that capturing most of the correct observations and the wrong observations. So for all those cases, recalls are high, but precision is low.

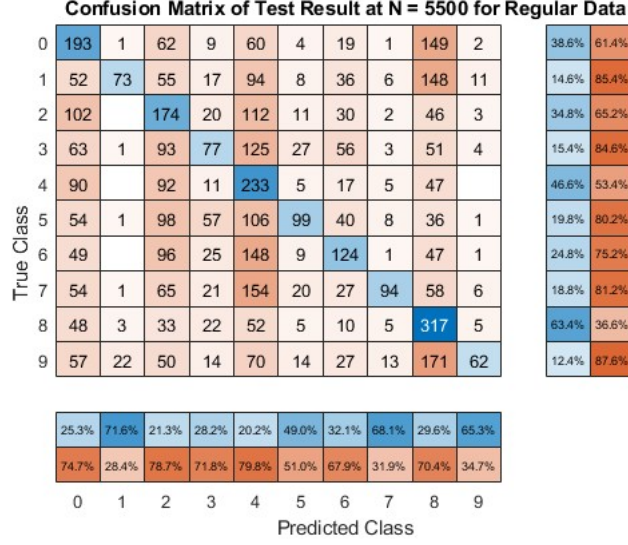


Figure 9: Confusion Matrix for kNN with Regular data where N=5500 and k=13

4. Principal Component Analysis

Before beginning the analysis, this paper gives a high-level explanation of Principal Component Analysis. PCA is a statistical technique used for data analysis and dimensionality reduction. The high-level idea behind it is to reduce the number of variables in the dataset while keeping as much of the variance or information as possible [4]. This process is beneficial when working with high-dimensional data.

To apply this technique in the project, the 'pca' function in Matlab is used. This function takes the raw data formatted in an n by p matrix and outputs six different results [4]. The first output is the 'coeff,' which are the principal component coefficients. It is the p by p matrix that contains weights or coefficients that represent the contribution of each of the features/variables or image pixels to the principal component. The next output is the 'score,' which is the original data in the principal component space [3]. Next is 'latent,' which is the principal component variance. The Hotelling's T-squared statistic for each observation in the dataset is returned in 'tsquared' [3]. The last two outputs are 'explained' and 'mu,' where the first is the total variance explained by each principal, and the second is the estimated mean of each variable in the dataset as used when centering [3]. The function, by default, uses single-value decomposition (SVD) as the principal component algorithm, but it can also use eigenvalue decomposition or alternating least squares algorithm [3]. This project sticks with SVD.

4.1. PCA Data Analysis

To begin the PCA analysis, the training data is passed into the function. Again this training data is set up with the same observations as those from the regular data analysis. The data used throughout all the different analyses are kept the same; the only difference is the method applied to the data for analysis. So this also uses the N value of 5500 to create the splits. Once the data is processed by the function a few things are plotted. The first and the last coefficients are examined to see how the patterns differ. As the plots do not add much information, they are only included in Appendix A.2 in Figure 20. But it is interesting to see that pattern in the first plot and the lack of a clear pattern in the second, as expected. Additionally, the variance each component explains is in Figure 21 in Appendix A.2. It takes about 26 components to explain 80 percent of the total variance in the dataset. This is the marker of total variance used for

dimensionality reduction in this project.

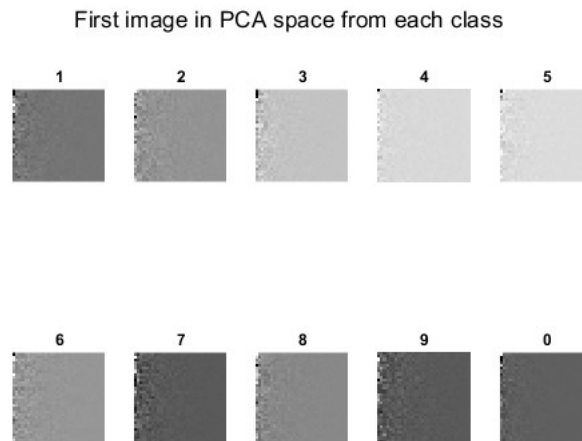


Figure 10: PCA Score

The next part of the analysis looks at the score or the data in the PCA embedding. Figure 10 shows the first image from each class in the new embedding. The main thing to note is that the features earlier have different values, while it starts to get consistent for the rest of the features. This aligns with the order of components based on how much total variance they explain. It is known that 80 percent of the variance is explained by the first 26 components. The reconstruction of the images to confirm the classes is in Figure 11. The images are exactly as seen in Figure 4. So not much information is lost in embedding to the PCA space.

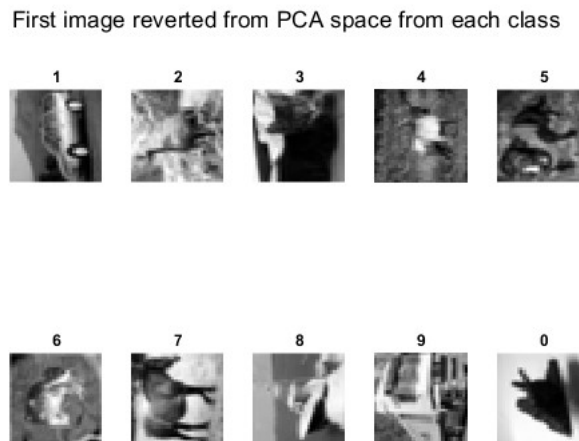


Figure 11: PCA Score Reconstructed

Another analysis that can be done with the PCA components is by plotting the first few components against each other. It can show if the data in the new embedding has helped separate the classes. Figure 12 shows the first and second components plotted against each other, and the data is still very much mixed up. The first two components explain a total of 43 percent of the

total variance in the data, but that is still not enough. Maybe with more components plotted, it would become more clear, but with just two, it is still not clear. Again this further emphasizes the importance of using all 26 components to get 80 percent of the total variance in the data.

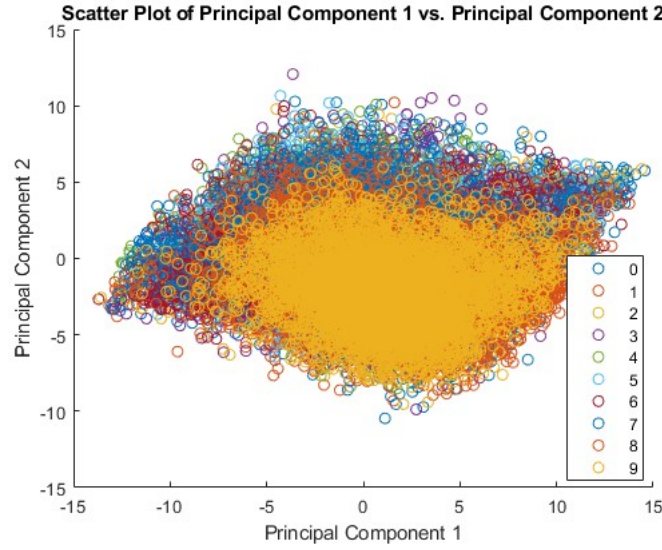


Figure 12: First and the Second Components of PCA Plotted Against Each Other

4.2. kNN with PCA Data and Results

The benefit of PCA comes from dimensionality reduction, so using the total variance marker of 80 percent and with that using only 26 of the components, a kNN model is created from the embedded data. The data splits are kept the same, with training and testing sets using the same observations used in the regular data kNN model. The validation set is not utilized in this analysis as there are not any hyperparameters that are being tuned. To set up the model, the score from the PCA is used to train, and the test set is embedded using Equation 1 and then used for testing. Other than that, nothing else is changed with the model in comparison to the regular data model, and class accuracy and global accuracy are used for analysis.

$$\text{score} = (\text{dataset} - \mu) * \text{coeff} \quad (1)$$

The results are shown in the confusion matrix in Figure 13. The global test accuracy is 0.3722, which is much better than that of the regular data. The class accuracy is listed in Figure 3.

Overall, the global accuracy is much better after dimensionality reduction. Looking further into the class accuracy as shown in Table 3, the best class is the again ship class (8) with the highest accuracy of 0.5600. After that, the frog class (6) climbed to second with an accuracy of 0.4980, followed by airplane (0) and bird (2) with an accuracy of 0.4480 and 0.4100, respectively. The deer class (4) scored lower, with an accuracy of 0.3980. The other classes, automobile (1), dog (5), horse (7), and truck (9), have similar accuracies above 0.25 and below 0.35, with the cat class (3) having the least accuracy of 0.1800. Most of the classes did better with PCA, but it seems that some did worse than before. Looking at the confusion matrix in Figure 13 gives a better explanation of what happened. The issue with certain classes being predicted the most seems to have improved slightly, and with that, the low precision should have slightly improved, with fewer false positives in most classes. Class 8 still seems to have both a lot of false positives and a lot of true positives but not as many false negatives. But again, the overall pattern is that all the classes are confused with each other, and the overall error is high, with lots of false

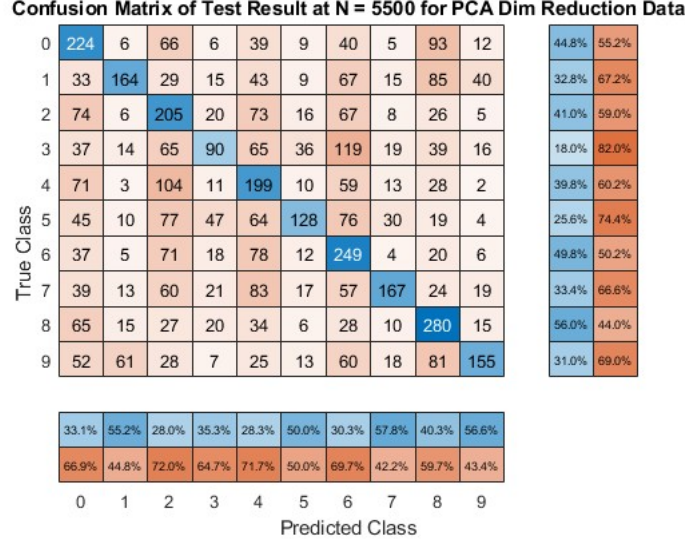


Figure 13: Confusion Matrix for kNN with PCA Dimension Reduced data where N=5500, k=13, and Using Only 26 Components

positives and false negatives. The classes that did the best, like 0, 2, 4, 6, and 8, were only scoring high due to the model thinking most of the observations were those classes, and with that capturing most of the correct observations and the wrong observations. This result is still an improvement upon the regular data.

5. Kernel Principal Component Analysis

Kernel Principal Component Analysis (kPCA) is a machine learning technique that extends the classical Principal Component Analysis (PCA) method, as seen earlier, to handle non-linear data. PCA works best for linearly separable data and may not perform well for more complex, non-linear data structures [8]. To address this limitation, kPCA uses a kernel function, which implicitly maps the original data into a higher-dimensional space, making it more linearly separable [8]. This allows the technique to capture complex, non-linear relationships among variables. Once the data is transformed, PCA is applied in the new space, resulting in principal components that better represent the structure of the original data. So basically, a transformation of data using the kernel and then PCA. With the transformation portion of the method comes a computationally heavy process.

To apply the method in this project, the 'KernelPCA' function is used [7]. This function is not a native function in Matlab but one created by the community. This library allows the user to pick different kernels with different parameters and returns outputs similar to that of the 'pca' function in Matlab. This library contains more functions and options than the PCA function in Matlab, but mostly the same outputs are used, for example, the score, variance explained, etc. [7]. To not overcomplicate the analysis, this project uses a Gaussian kernel with a gamma value of 1/1024. The gamma value is a default generally used by Python libraries. The values could be treated as hyper-parameters and adjusted to get the best performance, but this analysis is limited due to computational power and memory limits. Due to the limits, this method only uses a total of 20000 observations or 2000 observations from each class. If 20000 observations are used then the computer shuts down due to running out of memory or overheating. This is due to the kernel transformation process creating n by n matrix for the coefficients where n is the number of observations. This process is extremely memory hungry. With that, the kPCA method is applied

with only a third of the data used.

5.1. kPCA Data Analysis

Before beginning the analysis, the data setup for kPCA must be done. As no hyperparameters are being adjusted or tested, the dataset is split into a training and testing set. After understanding the limitations of the computer that only 20000 total observations can be used for kPCA, the analysis uses 2000 observations from each class. The training set uses 80 percent, and the testing set uses 20 percent. So, 16000 observations for the training set and 4000 for the testing set.

To begin the kPCA analysis, the training data is passed into the train portion of the kPCA model that does the analysis. Once the data is processed by the function a few things are plotted. The first and the last coefficients are examined to see how the patterns differ and are included in Appendix A.3 Figure 22. The plot shows all 16000 indices and their coefficients. So as stated earlier, while PCA uses features, kPCA components are created with the observations. It is again interesting to see all the active indices in the first plot and the lack of a clear pattern in the second. Additionally, the variance each component explains is in Figure 23 in Appendix A.3. It takes about 42 components to explain 80 percent of the total variance in the dataset. As that percentage is used in PCA, it is also used here.

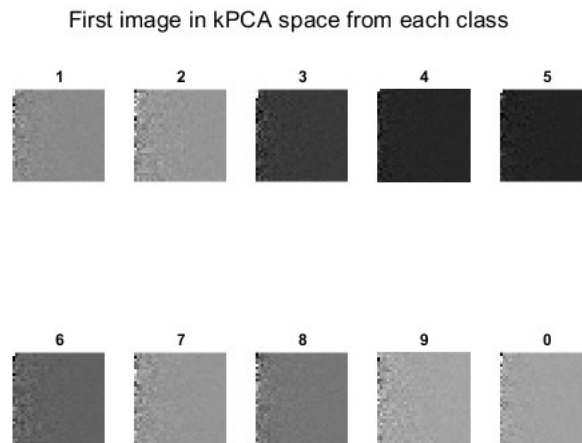


Figure 14: kPCA Score

The next part of the analysis looks at the score or the data in the kPCA embedding. Figure 14 shows the first image from each class in the new embedding. These look similar to that of the PCA score images. As with the PCA scores, the main thing to note is that the features earlier have different values, while it starts to get consistent for the rest of the features. This aligns with the order of components based on how much total variance they explain. The reconstruction of the images to confirm the classes is in Figure 15. Unlike the PCA score reconstruction, as seen in figure 11, the reconstruction from the kPCA data is hard to compare to Figure 4. All the images vary heavily. The nonlinear process which is used by kPCA is to blame for these differences.

With that, the components are also plotted against each other to see if the nonlinear process helped make the classes easier to separate by plotting the components against each other. Figure 16 shows the first and second components plotted against each other, and as with PCA, the data is still very much mixed up. The first two components explain a total of 40 percent of the total variance in the data, but that is still not enough. Again this further emphasizes the importance of using all 42 components to get 80 percent of the total variance in the data. Maybe with

First image reverted from kPCA space from each class

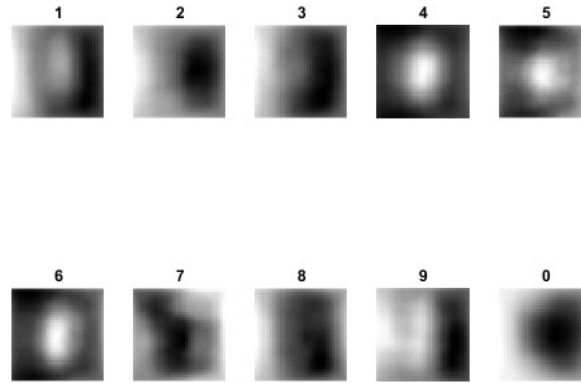


Figure 15: kPCA Score Reconstructed

components, the separation becomes more clear, but it is hard to plot all 42 components at the same time to visualize.

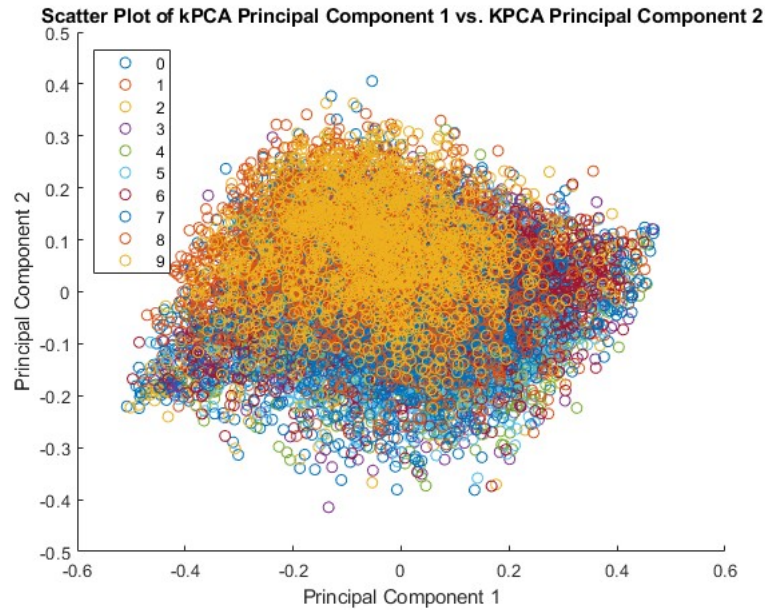


Figure 16: First and the Second Components of kPCA Plotted Against Each Other

5.2. kNN with kPCA Data and Results

Similar to PCA, the benefit of kPCA comes from dimensionality reduction, so using the total variance marker of 80 percent and with that using only 42 of the components, a kNN model is created from the embedded data. The training set is the same one created for the analysis, and the testing set is generated by passing the test data to the test function in the kPCA model, which creates the embedding. The validation set is not utilized in this analysis, as stated earlier,

as there are no hyperparameters that are being tuned. With that, the embedded training data is used to train the model, and the embedded testing set is used for testing. With the results, class accuracy and global accuracy are generated. The results of this model can not be compared to the previous methods as they have differing training and testing sets. At the end of the report, there is a section where all the methods are trained and tested with similar as used here, which is used for comparison.

The results are shown in the confusion matrix in Figure 17. The global test accuracy is 0.3160, which is still better than that of the regular data, where more data is used to train. The class accuracy is additionally listed in the Appendix A.3 Figure 4.

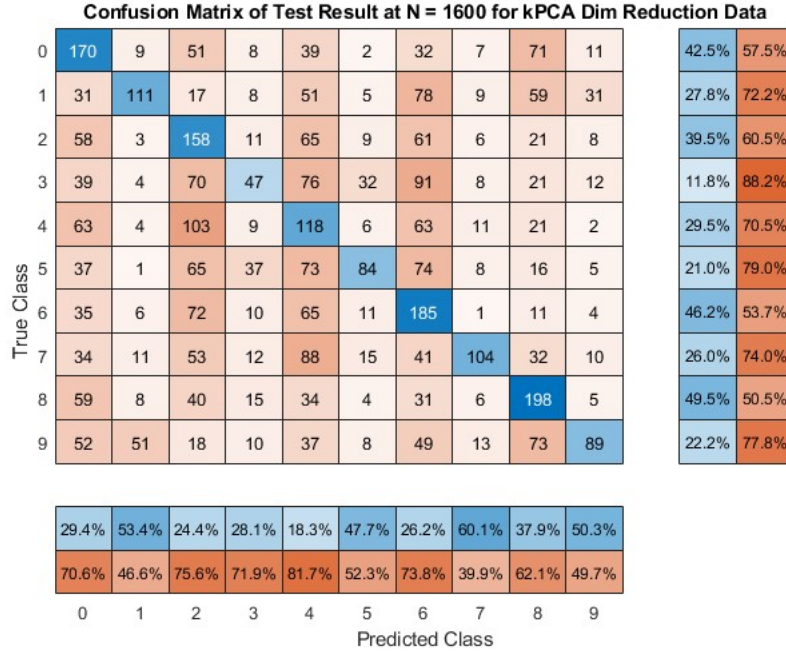


Figure 17: Confusion Matrix for kNN with kPCA Dimension Reduced data where N=1600, k=13, and Using Only 42 Components

Even with fewer observations used for training the model, the global test accuracy is better than that of the regular data. Looking further into the class accuracy as shown in Table 4, the best class is the again ship class (8) with the highest accuracy of 0.4950. Similar to PCA results, after that, the frog class (6) is second with an accuracy of 0.4625, followed by airplane (0) and bird (2) with an accuracy of 0.4250 and 0.3950, respectively. The other classes, automobile (1), deer (4), dog (5), horse (7), and truck (9), have similar accuracies between 0.2 and 0.3. with the cat class (3) having the least accuracy of 0.1175. The overall order in terms of class accuracy is similar to that of PCA. But as expected the accuracies are lower due to fewer amount of observations used to train the model. It is interesting that the overall accuracy is still better than that of the regular data. The class accuracies are most of better, but also some are lower. So using the same amount of data with the other representation methods, the comparison should year better conclusions.

Looking at the confusion matrix in Figure 17 gives a story almost exactly as that of the regular data confusion matrix in Figure 9 and worse than in Figure 13. Most of the classes that did well, like 0, 2, 4, and 8, only have a high score due to the model thinking most of the observations were those classes, and with that capturing most of the correct observations and the wrong observations, so with high recall and very low precision. Again, the overall pattern is that all the classes are confused with each other, and the overall error is high, with lots of false positives.

6. t-SNE

The final data representation method is t-Distributed Stochastic Neighbor Embedding (t-SNE). t-SNE is a machine learning algorithm used for dimensionality reduction and data visualization. Its primary objective is to preserve as much of the structure and relationships between data points as possible while representing high-dimensional data in lower-dimensional space, such as 2D or 3D [5], [9]. In its most basic form, t-SNE functions by identifying similarities between data points in the high-dimensional space and then attempting to reproduce these similarities in the lower-dimensional space [5], [9]. It minimizes the divergence between two probability distributions, one in high-dimensional space and the other in lower-dimensional space, and to effectively create visually comprehensible representations of complicated data, t-SNE maintains a balance between maintaining local structures (neighborhoods) and global structures (clusters) [5], [9].

In this project, the goal of t-SNE is to see if it could help visualize the data where the classes are separated. To apply the method in this project, the 'tsne' function built into Matlab is used [6]. The function takes in the data and spits out the embedding with two components. So basically turns the data down to 2D. Again, this method does not provide a mapping, just the embedding, so all the data must be passed in at once to get the embedding for both the training and testing set [6].

6.1. t-SNE Data Analysis

To begin the t-SNE data analysis, all the data is immediately passed into the function to get the embeddings. Once the embeddings are outputted, the data is then split into the proper training, testing, and validation sets sized similarly to what is done for the PCA and regular data. The validation set is again not used as there are not any hyperparameters that are being tuned. With the overall embeddings, the following analysis is done. One of the main points of using t-SNE is to visualize the high dimensional data in lower dimension or, in this case, 2D. Figure 18 show all of the embedding data plotted. Again, there are only two features in the embedding, so the data can be seen in 2D.

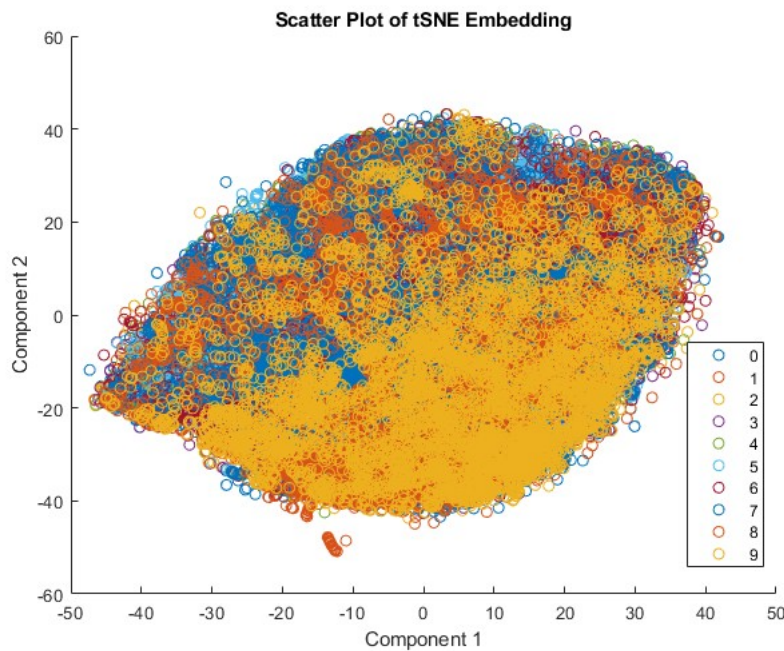
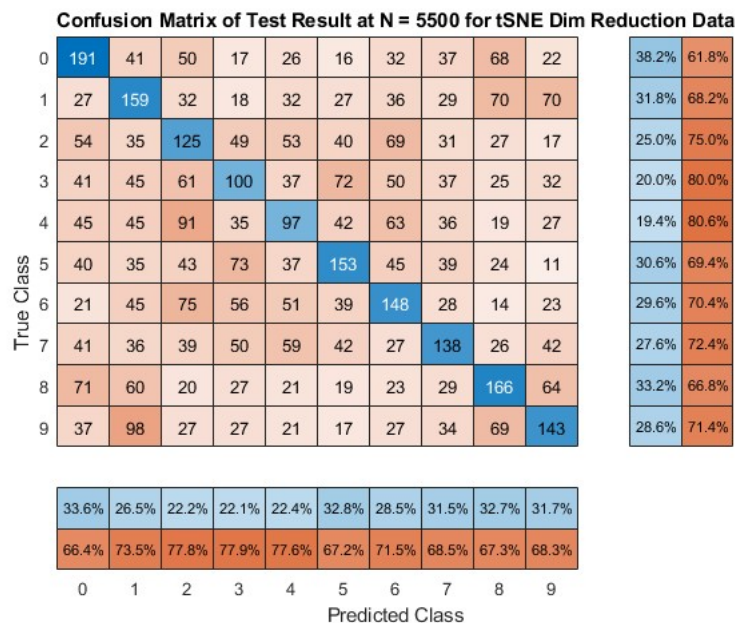


Figure 18: t-SNE Embedding Plotted

Figure 18 does, in fact, visualize the data, but it is clear that the data is not easy to separate by class. This plot can be compared to the other plots where the two components from PCA and kPCA are plotted, and the results are similar; all of the data points are very much mixed in, and the clusters of classes are hard to distinguish. The one interesting fact noticeable in the plot is while almost all the data points are clustered into the main large group, there is an outlier small cluster with most points from classes 6 and one from class 8. But overall, the results are as expected, and this overall shows the complexity of the dataset. The difficulty is all due to the background and other noise that cause the different images to be considered close, even if they are not the same classes [1]. The images here are further complicated due to the lack of color. While the Stanford note points out the similarity in color causing images to be close together, here, the lack of color further complicates it to differentiate the image, and it is all based on the intensity of the grayscale [1]. The effects of this should be visible in kNN as well, where the results might not be as good as desired.

With that, now the data is split into the train-val-test split, similar to how the regular data is. But as there are no hyperparameters to tune, only the train and test sets are used to confirm the test. The validation set is not utilized in this analysis, as stated earlier, as there are no hyperparameters that are being tuned, but it keeps the data consistent with the other methods. With that, the embedded training split of the data is used to train the model, and the embedded testing split of data is used for testing. With the results, class accuracy and global accuracy are generated.



Overall, the model seems to be performing worse regarding global accuracy. Looking further into the class accuracy as shown in Table 5, the results show something interesting. The global

accuracy might be low, but the lows are not as low, and the highs are not as high with respect to class accuracy compared to kPCA and PCA. The best class is now airplane class (0), with the highest accuracy of 0.3820. It is followed by the automobile (1), dog (0), and ship (8) with an accuracy of 0.3180, 0.3060, and 0.3320, respectively. Other classes like bird (2), cat (3), frog (6), horse (7), and truck (9) all had accuracy between 0.2 and 0.3. The least performing class was the deer class (4), with an accuracy of 0.1940. Overall, the accuracies average out to be lower than the overall accuracy of the regular data.

Looking at the confusion matrix in Figure 19 gives a very different story than any other confusion matrix in this project. The errors are spread out, and the errors seem to be higher with respect to the number of observations in the test set. One interesting thing in the plot is that classes 9 and 1 seem to be the ones that are confused the most, with both having most of each other as false positives. It does make sense as they are automobiles and trucks. Another interesting observation is that class 4 is incorrectly predicted most as class 2, which is deer being predicted as birds.

7. Comparative Analysis

In each of the analysis results explanation, some comparative analysis is done, but due to the issue arising from the kPCA model computation cost, the number of observations used is different for certain implementations. With that, now all the models must be adjusted to use the same amount of data with the same splits to do a thorough comparative analysis. This means the models only use the first 20000 observations or the first 2000 observations from each class. So, 1600 observations from each class are used for training and the remaining 400 for testing. With that, there are 16000 observations in the training set and 4000 in the testing. The validation set is ignored, as there is no hyperparameter tuning in this part of the project. With that, all the embeddings from the different data representation methods are formatted into the desired split and used to train and test kNN models. The results in terms of class and global accuracy are in Figure 1.

Class	Class Label	Regular Data	PCA Data	kPCA Data	t-SNE Data
0	Airplane	0.3700	0.4125	0.4250	0.2975
1	Automobile	0.1150	0.2825	0.2775	0.3000
2	Bird	0.3700	0.3900	0.3950	0.2075
3	Cat	0.1100	0.1500	0.1175	0.1600
4	Deer	0.3325	0.2975	0.2950	0.2000
5	Dog	0.1725	0.2100	0.2100	0.2900
6	Frog	0.2325	0.4375	0.4625	0.2125
7	Horse	0.1350	0.2750	0.2600	0.2825
8	Ship	0.5650	0.4875	0.4950	0.3700
9	Truck	0.1175	0.2225	0.2225	0.2575
Global/Average		0.2520	0.3165	0.3160	0.2578

Table 1: Global and Class Accuracy for Each Data Representation with N=1600 and k=13

Going in order of the class, each embedding has its advantages and disadvantages. The airplane class (0) has the greatest class accuracy with the kPCA embedding data. PCA and kPCA were fairly close, while t-SNE performed the worst. Interestingly, the Automobile class (1) has the best accuracy with t-SNE, while regular data did the worst. A pattern is already emerging. In almost all cases, kPCA and PCA perform similarly, while regular and t-SNE are more random. Bird (2) class is best classified by kPCA while worst by t-SNE. The cat class (3) is interesting in that t-SNE performed the best but kPCA and PCA do not perform similarly; kPCA performs close to regular data than PCA, and regular data performs the worst for the cat class. With the

deer class (4), the regular data performs the best, while t-SNE performs worst. The dog class (5) see the opposite result, with t-SNE performing the best while regular the worst. PCA and kPCA performed exactly the same. kPCA performs the best for Frog class (6), while t-SNE is the worst. Then again, t-SNE performs the best with horse class (7) and regular the worst. Finally, regular data performs the best for ship class (8), while t-SNE is the worst. With the final class truck (9), t-SNE performs the best while regular performs worst. In general, it is difficult to say which method is the best when looking at the class accuracy data. Five out of the ten classes are best classified by t-SNE, three by kPCA, and two by regular. Even though PCA does not perform the best in any of the classes it outperforms all of the methods regarding the average or global accuracy. The consistency with PCA is the main reason for its highest average accuracy. While it did not have the highest highs it also did not have the lowest lows. This helped PCA keep the average high.

Overall, PCA has the best accuracy with 0.3165, with kPCA fairly close at 0.3160, followed by t-SNE with 0.2578, and finally, regular data at last. So each method has its advantage, but looking back to the analysis with N at 5500, t-SNE did not perform as well as regular data. t-SNE is inherently doing dimensionality reduction, but the main purpose is visualizing high dimensional data rather than dimensionality reduction while the other methods like PCA and kPCA are mainly used for dimensionality reduction. With that, PCA and kPCA for sure perform much better than regular data. With further adjustment and fine-tuning of the kernel function and parameters, kPCA could potentially perform better than regular PCA, given that this is a nonlinear dataset. Additionally, when dealing with low accuracy, noise can heavily affect the results, and with this dataset, that is a serious obstacle. So, using a better-fit classifier like neural net models, the results might be much better, but that requires further analysis.

8. Conclusion

In conclusion, each data representation method, including regular data, has its own set of advantages and disadvantages. The dataset used in this study effectively assesses these methods, given the complexity. Although kNN may not be the most suitable classifier for this dataset, employing various representation techniques has led to slight improvements in performance. PCA, which is less computationally demanding, emerges as the best overall performer. However, kPCA offers the potential for better results through further fine-tuning, such as optimizing the kernel function and associated parameters. Although not explored in this project due to computational constraints, further tuning could lead to better outcomes than PCA. On the other hand, the t-SNE method is mainly utilized for visualization purposes. By examining the plot it generates, it becomes evident that its performance is somewhat limited with respect to this dataset, as it only had two features to store all the variance. Interestingly, class performance analysis reveals that all methods had instances of achieving the best performance, reinforcing the need for an additional examination of these methods using the complete set of data features to validate the results. Taking into account the implementation/execution methods and computational cost associated with the grayscale data, PCA proves to be the most effective method to employ in this context. The next step is to use the color data and further compare the results with fine-tuning involved.

References

- [1] “Image Classification,” 2023. [Online]. Available: <https://cs231n.github.io/classification/>
- [2] A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” 2009. [Online]. Available: <https://www.cs.toronto.edu/~kriz/cifar.html>
- [3] “pca,” *MathWorks*, 2023. [Online]. Available: <https://www.mathworks.com/help/stats/pca.html>
- [4] “Principal Component Analysis (PCA),” *MathWorks*, 2023. [Online]. Available: <https://www.mathworks.com/help/stats/principal-component-analysis-pca.html>
- [5] “t-SNE,” *MathWorks*, 2023. [Online]. Available: <https://www.mathworks.com/help/stats/t-sne.html>
- [6] “tsne,” *MathWorks*, 2023. [Online]. Available: <https://www.mathworks.com/help/stats/tsne.html#d124e985536>
- [7] “Kernel Principal Component Analysis (KPCA),” *MathWorks*, 2023. [Online]. Available: <https://www.mathworks.com/matlabcentral/fileexchange/69378-kernel-principal-component-analysis-kpca>
- [8] D. Pelliccia, “PCA and kernel PCA explained,” *NIRPY Research*, 2020. [Online]. Available: <https://nirpyresearch.com/pca-kernel-pca-explained/>
- [9] M. Lee, “What is tSNE?,” *Sonrai Analytics*, 2021. [Online]. Available: <https://sonraianalytics.com/what-is-tsne/>

A. Appendix

This section stores any tables or plots that are not fully needed for the analysis or are already described in the analysis. For example, all the class accuracy values are already described in the analysis. The coefficient plots are not needed for the analysis and are just extra analysis done to just compare results with previous projects. The cumulative sum plots of variance explained are just to see if desired, and the main marker at 80 percent explained is already stated in the analysis.

A.1. Regular Data Extra Figures and Tables

Class	Accuracy
0	0.3860
1	0.1460
2	0.3480
3	0.1540
4	0.4660
5	0.1980
6	0.2480
7	0.1880
8	0.6340
9	0.1240

Table 2: Class Test Accuracy with $N=5500$ and $k=13$ for Regular Data

A.2. PCA Extra Figures and Tables

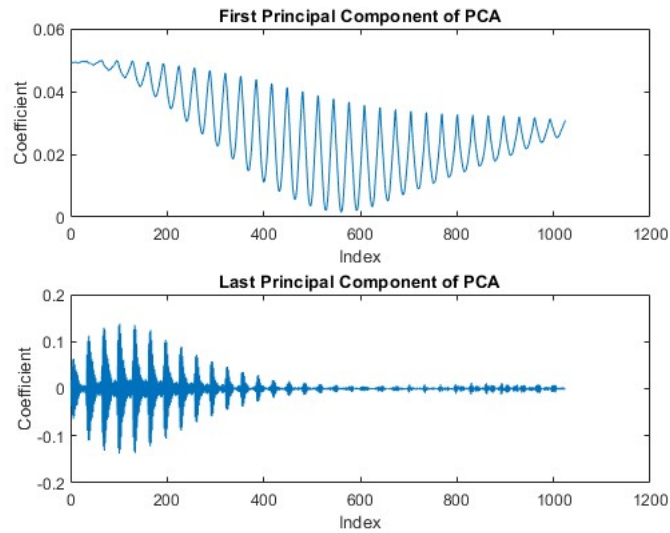


Figure 20: First and Last Coefficients of PCA

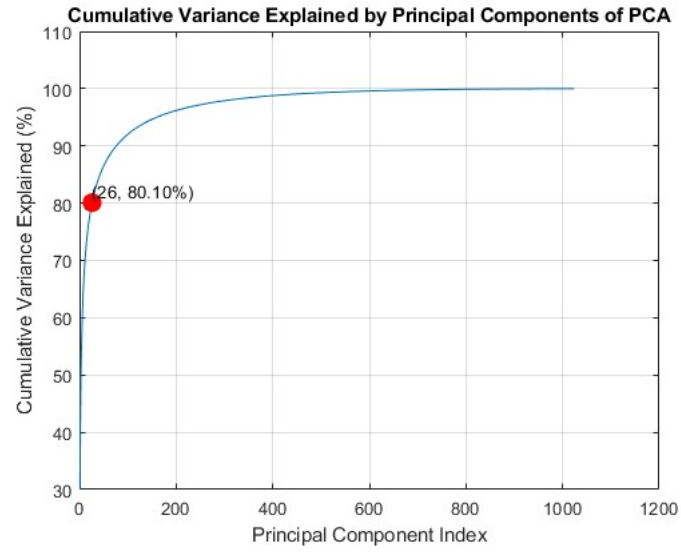


Figure 21: PCA Components and the Cumulative Sum of How Much of the Total Variance Each Component Explains

Class	Accuracy
0	0.4480
1	0.3280
2	0.4100
3	0.1800
4	0.3980
5	0.2560
6	0.4980
7	0.3340
8	0.5600
9	0.3100

Table 3: Class Test Accuracy with N=5500 and k=13 for PCA Data

A.3. kPCA Extra Figures and Tables

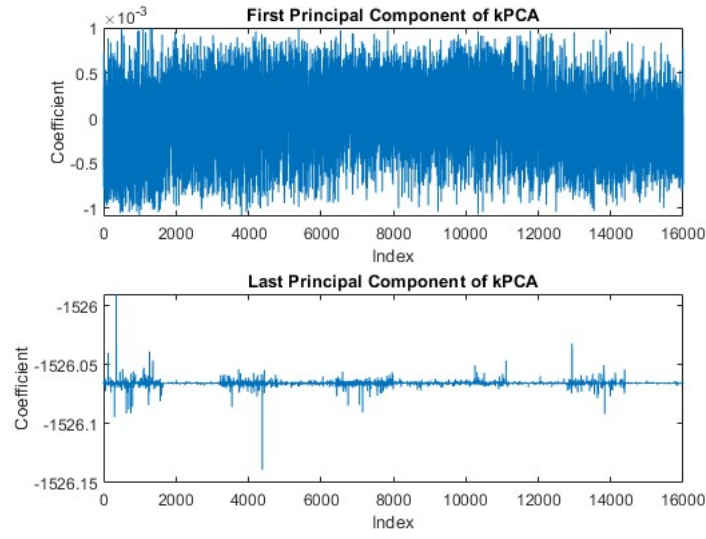


Figure 22: First and Last Coefficients of kPCA

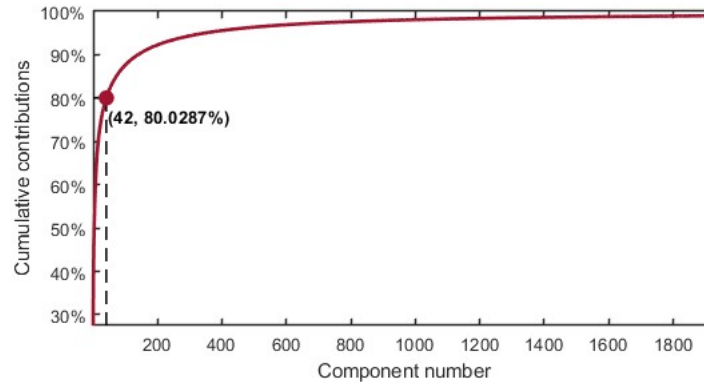


Figure 23: kPCA Components and the Cumulative Sum of How Much of the Total Variance Each Component Explains

Class	Accuracy
0	0.4250
1	0.2775
2	0.3950
3	0.1175
4	0.2950
5	0.2100
6	0.4625
7	0.2600
8	0.4950
9	0.2225

Table 4: Class Test Accuracy with N=1600 and k=13 for kPCA Data

A.4. t-SNE Extra Figures and Tables

Class	Accuracy
0	0.3820
1	0.3180
2	0.2500
3	0.2000
4	0.1940
5	0.3060
6	0.2960
7	0.2760
8	0.3320
9	0.2860

Table 5: Class Test Accuracy with N=5500 and k=13 for t-SNE Data